

LLD Case Study 8: In-Memory API Rate Limiter

Target Patterns: Strategy Pattern, Encapsulation, State Synchronization
Key Engineering Focus: High-throughput rate-limiting algorithms: Token Bucket, Leaky Bucket, and Sliding Window Log with thread-safe client isolation.

1. Problem Statement & Functional Requirements

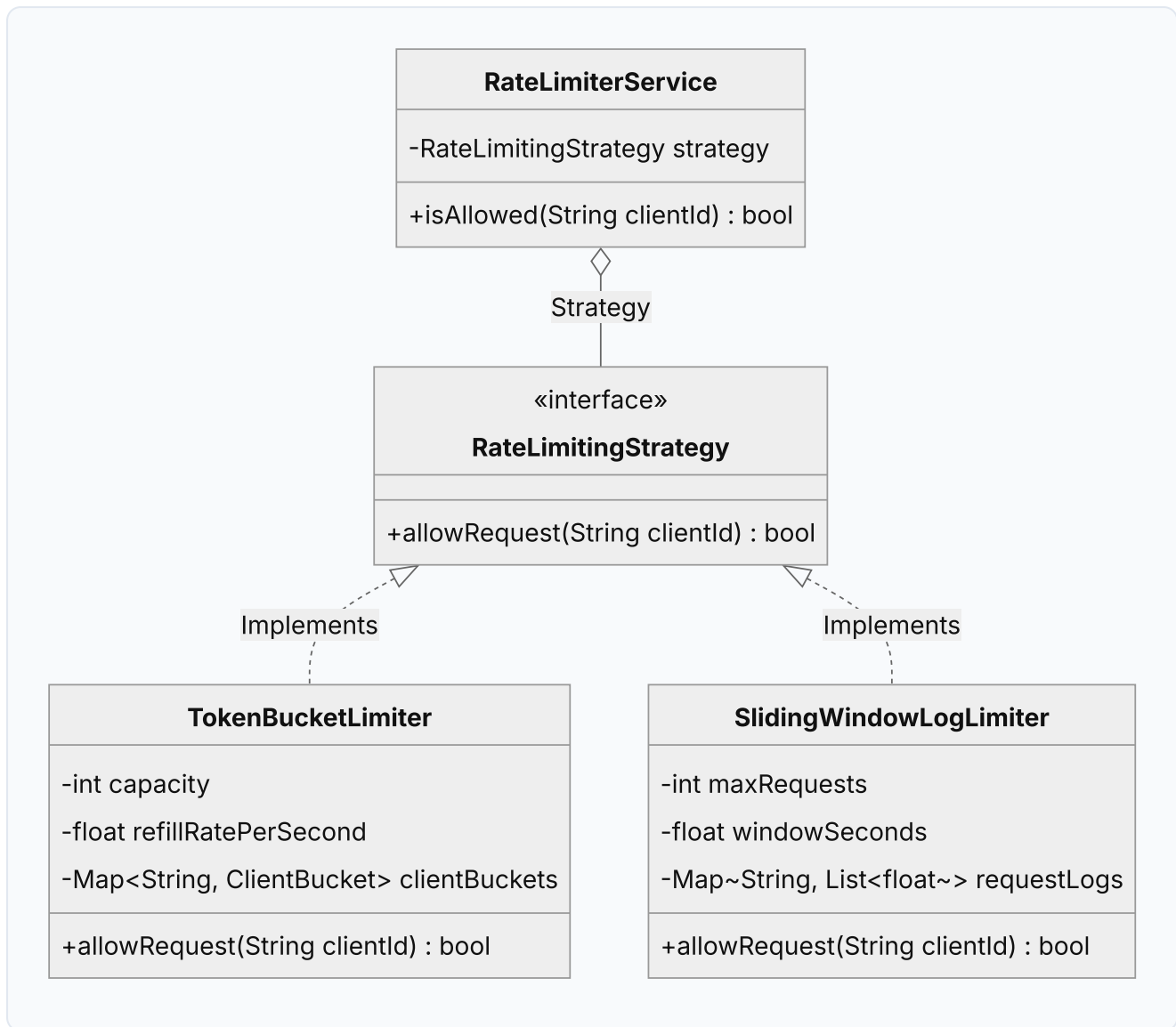
Design an API rate limiter capable of protecting backend microservices from Denial-of-Service (DoS) attacks, brute-force requests, and resource starvation.

Requirements:

- 1. Pluggable Algorithms (Strategy Pattern):** Easily interchange between **Token Bucket**, **Leaky Bucket**, and **Sliding Window Log**.
- 2. Client Isolation:** Maintain independent rate limits per Client ID / IP address.
- 3. Thread-Safety:** Atomic token consumption under high concurrent request volume.
- 4. Instant Evaluation:** Must execute in under 1 millisecond ($O(1)$ time complexity).

2. Algorithm Comparison & Trade-Offs

Algorithm	Memory Consumption	Burst Handling	Accuracy	Use Case
Token Bucket	$O(1)$ per client	Supports bursts up to bucket capacity	High	General API Gateways (Stripe, GitHub)
Leaky Bucket	$O(\text{Capacity})$ buffer	Enforces smooth, fixed outflow rate	High	Egress traffic shaping
Sliding Window Log	$O(N)$ requests in window	Accurate without boundary resets	100% Exact	Strict financial APIs



3. Production-Grade Python Implementation



4. Edge Cases, Tests and Extensions

Edge cases the code above handles, and the ones it does not

Case	What happens today	What a production system does
Clock jumps (NTP step, VM pause)	Safe: <code>time.monotonic()</code> never goes backwards	Never use <code>time.time()</code> for elapsed time
Refill after a long idle period	<code>min(capacity, ...)</code> caps the bucket, so no hoarded burst	Same cap; document the burst size to clients
Two threads, same client	Per-bucket lock makes consume atomic	Same, or an atomic Redis Lua script when distributed
Millions of one-off client IDs	Leak: <code>buckets</code> grows forever	Evict buckets idle longer than <code>capacity / refill_rate</code> seconds (a full bucket is indistinguishable from a new one)
Several app servers	Each keeps its own count , so the real limit is <code>N x limit</code>	Central store (Redis <code>EVALSHA</code>) or sticky routing by client key
Rejected request	Returns <code>False</code> only	Return <code>Retry-After</code> computed from the refill rate

Deterministic tests (no sleeping)

Real sleeps make rate-limiter tests slow and flaky. Patch the clock instead. This block extends the implementation above and runs as is.

Extensions interviewers ask for

- Tiered limits (free vs paid):** make the strategy factory take a plan, keep one limiter per plan, and key buckets by `(plan, client_id)`.
- Return remaining quota and `Retry-After`:** `consume()` already knows `tokens`; return `(allowed, tokens_left, seconds_until_next_token)` and map it to `X-RateLimit-*` headers.
- Distributed version:** move the bucket state to Redis and run the refill-and-consume as one Lua script so it stays atomic; fail open or closed on Redis outage depending on whether you protect a backend (fail closed) or user experience (fail open).
- Sliding window counter:** keep two fixed-window counters and weight the previous one; this approximates the log's accuracy at $O(1)$ memory.

Follow-up questions

- *Why is a token bucket allowed to burst while a leaky bucket is not?* The bucket stores unused capacity as tokens, so an idle client can spend `capacity` requests at once. A leaky bucket drains at a fixed rate regardless of history.
- *What is the memory cost of the sliding window log at 10,000 requests per client per window?* One timestamp per request, so about 10,000 floats per client; that is why high-volume APIs use the counter approximation.
- *Where would you put the limiter: client, gateway or service?* At the gateway for coarse protection, plus a per-service limiter for resources only that service knows are expensive; never rely on the client.